QSpice Schematic and Symbol File Formats

An unofficial reference to .qsch and .qsym files

The QParser2 Project
Version 0.1 — 2026-10-18

About this document 2
Overview 3
File structure 3
- Bytes on disk 3
- The file header 4
- Records and lines 4
- Keywords 5

Record order 5
Field types 6
- Points 6
- Coordinates and units 6
- Integers, hexadecimal, and decimals 6
- True and false 6
- Strings 6
- Text encoding 7
- Line breaks in text 7
- Overbars 7

Shared fields 8
- Rotation code 8
- Text rotation and alignment 8
- Colors 9
- Line width 10
- Line type 11
- Font size 11
- Lookup index and pin index 11

Containers 11
- Schematic 11
- Component 12
- Symbol 13

Symbol properties 13
- type: 13
- description: 14
- library file: 14
- shorted pins: 14

Electrical records 15
Net 15
Junction 16
Wire 16
Tap 16
Pin 17
Graphic records 18
Line 19
Box-defined shapes 19
Rect 19
Image rectangles 20
Ellipse 20
Arc3p 20
Arc 21
Triangle 21
Coil 21
Zigzag 22
Text 22
On a schematic 22
In a symbol 23
Connectivity 23
Choosing a label's alignment 24
Hierarchical blocks 24
Symbol files 24
Where records may appear 25
Copying and pasting 25
Open questions 25
Appendix A: A complete example 26
Appendix B: Record summary 27
Appendix C: Sources 28

About this document

This is not official documentation. QSpice is a product of Qorvo, Inc. Qorvo has not published a specification of its file formats, and this document was not written, reviewed, or endorsed by Qorvo. Questions about it go to its author, not to Qorvo.

Everything here was worked out by reading files that QSpice wrote, by changing files, watching what QSpice did with them, and by comparing drawings side by side. It is what The QParser2 Project's code (QParser2_Shared_Lib, the QSch_Shared_Libs, QSchEdit, and QSchView) relies on to read, draw, and write these files. It may be incomplete or wrong, and a later QSpice may change the format without notice.

Use it at your own risk:

- Keep a copy of any file before writing to it with your own tools.

- Check what your program writes by opening the result in QSpice.
- Treat anything marked *unknown* or *inferred* as a guess.

How sure we are. Statements carry one of three levels where it matters:

Level	Meaning
Confirmed	Measured against files or drawings QSpice produced.
Inferred	Consistent with everything seen, but not tested directly.
Unknown	Not measured. A note says what is known.

Statements with no mark are confirmed.

Corrections. The QParser2 Project is at https://github.com/robdunn4/QSpice under the GPL-3.0 license. Corrections and new findings are welcome on Qorvo's QSpice forum (https://forum.qorvo.com/c/qspice/9), addressed to @RDunn.

Overview

QSpice uses one text-like format for two kinds of file:

File	Holds	Root record
`.qsch`	A schematic: Components, wires, net labels, graphics, text, and SPICE directives.	`«schematic»`
`.qsym`	One symbol: Its drawing, pins, attribute text, and properties.	`«symbol»`

A file is a **tree of records**. Each record is one line: A keyword, then its fields. Some records are containers and hold other records on the lines that follow.

Two things are worth knowing from the start:

- **Every component carries its whole symbol.** A component in a schematic is a position and an orientation, wrapped around a complete copy of its symbol. The schematic does not refer to the `.qsym` it came from, so a schematic opens the same on a computer that has none of its symbol files.
- **Order matters.** Records of each kind are grouped, the groups come in a fixed order, and several fields refer to other records by their position within a group. See *Record order*.

File structure

Bytes on disk

A file is a sequence of bytes, not Unicode text. The structure uses three single bytes that are not ASCII:

Byte	As Windows-1252	Purpose
`0xAB`	«	Begins a record.
`0xBB`	»	Ends a record.
`0xFF 0xD8 0xFF 0xDB`	(four bytes)	The file header.

These are single bytes. A program that reads the file as UTF-8 text will see invalid sequences, and one that writes « and » as UTF-8 (two bytes each) produces a file QSpice cannot read. Read and write the file as bytes, and decode only the text fields, one by one (see *Text encoding*).

The file header

A file begins with the four bytes `FF D8 FF DB`. QSpice writes them on every `.qsch` and every `.qsym` it saves.

The symbol files installed with QSpice have **no** header: They begin directly with `«symbol`. Our programs therefore require the header on a `.qsch` but accept a `.qsym` without it. They always write it.

Records and lines

Each record is one line, ending in a line feed (`0x0A`). QSpice writes LF only; a reader should also accept CR LF.

A **leaf** record begins with « and ends with » on the same line:

```
«junction (1500,0)»
```

A **container** record begins with « and has no » on its line. The records it holds follow, one per line, and a line holding only » closes it:

```
«component (-1000,1000) 0 0
  «symbol R
    ...
  »
»
```

Lines are indented by two spaces per level of nesting. The indentation is for people: A reader should ignore leading spaces.

QSpice usually ends a file with a blank line after the final ». A reader should ignore blank lines.

In grammar form:

```
file      := header? item
item      := leaf | container
leaf      := INDENT "«" keyword (" " fields)? "»" LF
container := INDENT "«" keyword (" " fields)? LF  item*  INDENT "»" LF
```

A line is a leaf when its last byte is ». Every other line that begins with « opens a container.

Keywords

A record begins with its keyword. A keyword may contain spaces and colons (`library file:`). It is followed by a single space and then the fields, or by nothing.

Keywords are matched exactly, including case. The complete list is in *Record order*.

Record order

Inside any container, records are **grouped by kind**, and the groups always come in this order. QSpice writes them this way, and our programs sort to it on reading.

Order	Keyword	Kind	Container?
1	`schematic`	Schematic root	Yes
2	`component`	Placed component	Yes
3	`symbol`	Symbol	Yes
4	`type:`	Symbol property	No
5	`description:`	Symbol property	No
6	`library file:`	Symbol property	No
7	`shorted pins:`	Symbol property	No
8	`net`	Net label, port, or ground	No
9	`junction`	Junction dot	No
10	`wire`	Wire or bus	No
11	`tap`	Bus tap	No
12	`line`	Graphic line	No
13	`rect`	Rectangle or image	No
14	`ellipse`	Ellipse	No
15	`arc3p`	Three-point arc	No
16	`arc`	Four-point arc	No
17	`triangle`	Triangle	No
18	`coil`	Coil (inductor loops)	No
19	`zigzag`	Zigzag (resistor)	No
20	`text`	Text, directive, or attribute	No
21	`pin`	Symbol pin	No

Inferred: QSpice may depend on this order. Our programs never write anything else, and we have not tested what QSpice does with a file whose groups are out of order.

Order within a group matters too. Several things are identified by position:

- A symbol's pins are numbered by the order of their `pin` records. The first is pin 1 in the netlist.
- In a symbol, the first `text` record is the reference designator and the second is the value (see *Text*).
- The lookup index and pin index fields refer to a `text` or `pin` record by its zero-based

position within its group (see *Lookup index and pin index*).

A tool that inserts, deletes, or reorders records must keep these references right.

Field types

Fields are separated by single spaces. Each record has a fixed list of fields, with an optional field or two at the end of some (given with each record).

Points

A point is written `(x,y)`: Two integers in parentheses, separated by a comma, with no spaces. For example, `(-1700,-500)`.

Coordinates and units

Coordinates are **integers**. There are no fractional coordinates anywhere in the format.

The Y axis points up. A larger y is higher on the page, as on a graph, so a resistor's pin at `(0,200)` is above one at `(0,-200)`.

The grid. QSpice places electrical things (pins, wire ends, net labels, and component positions) on a 100-unit grid. Graphics need not be on any grid. *Unknown:* How finely QSpice's editor snaps graphics.

Size. The file gives no physical size. Our programs print and export 1000 units as one inch (25.4 mm), which suits QSpice's symbols. That is our convention, not something the file states.

Integers, hexadecimal, and decimals

- **Integers** are plain decimal, with a minus sign where needed: `0`, `-1`, `13`.
- **Hexadecimal** values (colors and some flags) begin with `0x` and use lowercase digits: `0x1000000`, `0xff0000`. Leading zeros are not written.
- **Decimals** (font sizes) are written as short as possible: `1`, `0.681`, `1.25`.

True and false

The words `true` and `false`, in lowercase.

Strings

Most strings are enclosed in double quotes: `"R1"`, `".tran 10m"`. The quotes are part of the stored field and are not escaped inside: QSpice gives no way to put a double quote inside a quoted string, and a string containing one may not read back correctly.

A quoted string is always the **last** field of its record. A reader takes everything from the opening

quote to the end of the line, not to the next quote, so a string may contain spaces.

Two records end with **two** quoted strings, the second optional: `net` (name, then description) and `pin` (label, then net name). Names take no spaces, so a reader separates the two at the first space.

A few records hold a bare, unquoted string. For `type:`, `description:`, and `library file:`, it is everything after the keyword and its space, to the end of the line. A `symbol` record's name is one word, with no spaces.

Text encoding

Text fields are bytes, and QSpice uses **two encodings** for them:

Form	**When QSpice uses it**
Windows-1252, one byte per character	Short text in which every character fits, such as a µ in a capacitor's value.
UTF-8, preceded by a byte-order mark (`EF BB BF`)	Text with characters outside Windows-1252, and longer typed or pasted text such as SPICE directives.

The byte-order mark comes **after** the opening quote:

```
«text (4714,13230) 1 15 1 0x1000000 -1 -1 "ï»¿BOOST CURRENT"»
```

(The `ï»¿` there is the byte-order mark, shown as Windows-1252.)

To **read** a text field: Try UTF-8; if the bytes are not valid UTF-8, read them as Windows-1252. Remove any byte-order mark either way. Windows-1252 differs from Latin-1 in bytes `0x80` to `0x9F`, which include the euro sign, so Latin-1 is not a safe substitute.

To **write** one: Use Windows-1252 if every character fits, and UTF-8 with the byte-order mark otherwise. Text with an overbar marker is always written as UTF-8 with the mark (see *Overbars*). UTF-8 *without* the mark is neither of QSpice's forms, and text written that way may not read back as written.

Line breaks in text

A record is one line, so a line break inside text is stored as the two characters backslash and `n`. Any `\n` in a text field is read as a line break, including one the user typed on purpose.

Overbars

A character drawn with a bar over it (an active-low signal name, for example) is preceded by the marker character ¬ (U+00AC). Each barred character has its own marker. For example, "Overbar Text" with every character barred, including the space, is stored as:

```
"¬O¬v¬e¬r¬b¬a¬r¬ ¬T¬e¬x¬t"
```

The markers are part of the text. In net names this means that **Q and barred Q are different nets**.

Unknown: QSpice saves overbarred text as UTF-8 with the byte-order mark, where ¬ is two bytes. Whether it reads a single Windows-1252 byte `0xAC` as a marker has not been tested.

Shared fields

These fields appear in many records and mean the same thing in each.

Rotation code

Components and the box-defined graphics (`rect`, `ellipse`, `arc`, `coil`, and `zigzag`) carry a rotation code from 0 to 15:

Bits	Meaning
0–2	Rotation, in steps of 45° counter-clockwise: 0 = 0°, 2 = 90°, 4 = 180°, 6 = 270°.
3 (value 8)	Mirror: The X axis is negated after rotating.

For a component, the code transforms the symbol's own coordinates. A symbol point (x, y) lands at the component's position plus M × (x, y), where M is:

Code	M	Result
0	(1, 0; 0, 1)	Upright
1	(1, −1; 1, 1)	45°, and enlarged
2	(0, −1; 1, 0)	90°
3	(−1, −1; 1, −1)	135°, and enlarged
4	(−1, 0; 0, −1)	180°
5	(−1, 1; −1, −1)	225°, and enlarged
6	(0, 1; −1, 0)	270°
7	(1, 1; −1, 1)	315°, and enlarged

Each matrix is written (row 1; row 2), so $x' = m11 \cdot x + m12 \cdot y$ and $y' = m21 \cdot x + m22 \cdot y$. With the mirror bit set, negate the first row (x′) after applying the matrix.

Odd codes are not true 45° rotations. They rotate by 45° and also enlarge by √2, about 41%. The matrices hold only 0, 1, and −1, so a pin on the 100-unit grid stays on it, which a true 45° rotation could not do. QSpice draws such a symbol 41% larger.

QSpice writes an odd code when a component's rotation is set to a 45° angle: Setting −45° writes code 7. A 45° turn changes only the code; the symbol's records stay the same.

For a box-defined graphic, the code is described with those records (see *Box-defined shapes*).

Text rotation and alignment

`text`, `net`, and `pin` records carry one integer that packs the text's alignment and rotation:

Bits	Meaning
0–1	Vertical alignment: 1 = north (top), 2 =

Bits	Meaning
	south (bottom), 3 = center.
2–3	Horizontal alignment: 1 = west (left), 2 = east (right), 3 = center.
4–7	Rotation, in steps of 45° counter-clockwise.

The alignment says **which point of the text sits on its anchor**. For example, 7 (west, center) puts the left end of the text on the anchor, so the text extends to the right of it.

Value	Binary	Anchor sits on the text's
0	0000	Center (see below).
5	0101	Top-left corner.
6	0110	Bottom-left corner.
7	0111	Left edge, centered.
9	1001	Top-right corner.
10	1010	Bottom-right corner.
11	1011	Right edge, centered.
13	1101	Top edge, centered.
14	1110	Bottom edge, centered.
15	1111	Center.

The rotation sits in the upper four bits, so a value is rotation × 16 + alignment. For example, 46 is rotation 2 (90°) with alignment 14 (bottom center), and 110 is rotation 6 (270°) with alignment 14.

Alignment 0. QSpice writes 0 in some places:

- On a `text` record, it draws as centered both ways. (*Inferred*)
- On a `net` label, it means a label anchored where several wires cross, drawn centered.
- On a `pin`, it hides the pin's label (see *Pin*).

Unknown: The format allows rotations 0 to 15. Only 0 to 7 have been seen.

Colors

Colors are hexadecimal: `0xMMBBGGRR`, with a **mode** in the top byte and blue, green, and red in the lower three (blue first, as Windows stores colors).

Mode	Meaning as a line or text color	Meaning as a fill
`0x00`	The RGB color in the lower bytes.	The RGB color in the lower bytes.
`0x01`	The default color for this kind of element, from QSpice's settings.	No fill (hollow).
`0x02`	*Unknown:* Never seen as a line color.	*Inferred:* The background color. Seen on triangles; its meaning has not been confirmed.

Mode	Meaning as a line or text color	Meaning as a fill
`0x03`	*Unknown:* Never seen as a line color.	Solid: The voltage source's + and − bars, the NPN transistor's arrowhead. *Inferred:* In the drawing color (symbol blue). It may instead be the shape's own line color; no file seen tells the two apart.
`0x04`	The default color, the same as `0x01`.	The "solid annotation" fill color from QSpice's settings (yellow by default).
`0x05`–`0x0B`	*Unknown.*	An image, drawn with a Windows raster operation (see *Image rectangles*).

The image modes, in order, are SRCCOPY, SRCPAINT, SRCAND, SRCINVERT, SRCERASE, NOTSRCERASE, and MERGEPAINT.

With modes `0x01` to `0x04`, the RGB bytes are usually filled in too, apparently copied from QSpice's settings when the color was chosen. Our programs ignore them and use the current settings, so a change to QSpice's colors restyles existing drawings. *Unknown:* Whether QSpice uses the stored RGB bytes or its current settings.

Common values:

Value	Meaning
`0x1000000`	Default color, or no fill.
`0x0`	Black (mode 0). QSpice writes it on the pins of its own symbols, whose labels are hidden.
`0xff0000`	Blue (BB = ff).
`0x3000000`	Solid fill in the drawing color.
`0x4000000`	Solid annotation fill.

Line width

An integer. Width 0 means **the default**: The "CAD line width" from QSpice's settings, 1.5 unless the user changed it. Any other width w draws a line **w + 1 units thick**, in schematic units, so lines grow thicker as the view zooms in.

Width	Thickness
0	Default (1.5 + 1 = 2.5 units, unless changed)
1	2 units
2	3 units
7	8 units

Unknown: Inside a symbol drawn at an odd rotation code, whether QSpice enlarges line

thickness by √2 with the rest of the drawing.

Line type

An integer giving the Windows pen style. QSpice offers these five:

Type	Style
0	Solid
1	Dashed
2	Dotted
3	Dash-dot
4	Dash-dot-dot

Font size

A decimal. QSpice's menus offer 0.65, 0.8, 1, 1.25, 1.6, 2, 2.5, 3, 4, and 5, and other values also work. Pins use other sizes too: QSpice's own symbols include 0.681.

QSpice draws schematic text in **Tahoma**. The height of capital letters is proportional to the font size: About 116 units per 1.0 of font size, measured on the font's cap line (about 121 units if measured from the darkest pixels of rendered text). At font size 1, capitals are a little taller than one 100-unit grid step.

Lookup index and pin index

Graphic and text records inside a symbol end with one or two indexes. Each is −1 when unused.

Lookup index. When not −1, a zero-based index into the symbol's `text` records. It marks a record that QSpice merged into the symbol through the LOOKUP programmable attribute, and points to the `text` record holding that attribute. *Unknown:* LOOKUP itself has not been studied in detail.

Pin index. When not −1, a zero-based index into the symbol's `pin` records. It associates a graphic with a pin; QSpice's symbol editor sets it with "Associated Pin". *Inferred:* A graphic associated with a pin that has been disabled (see *Pin*) is hidden along with the pin.

Because both refer to records by position, a tool that adds, removes, or reorders `text` or `pin` records must correct every index that refers to them.

Containers

Schematic

```
«schematic
  ...
»
```

The root of a `.qsch` file. It has no fields.

A schematic holds `component`, `net`, `junction`, `wire`, `tap`, graphic, and `text` records. It does not hold `pin` records or bare `symbol` records (see *Where records may appear*).

Component

```
«component (x,y) rotation stuffing
  «symbol ...»
»
```

A placed symbol. It holds exactly one `symbol` record: A complete copy of the symbol, in the symbol's own coordinates.

Field	Type	Meaning
(x,y)	Point	Where the symbol's origin is placed.
Rotation	Integer	Orientation of the symbol, from 0 to 15 (see *Rotation code*).
Stuffing	Integer	0 = fitted; 1 = do not stuff (or a disabled block); 2 = replaced by a jumper (short).

A component that is not stuffed, or is replaced by a jumper, is still drawn. Our programs draw it gray, with the jumper between its two pins in the wire color. *Unknown:* Exactly how QSpice draws it.

An example from a QSpice file, a resistor at (−1000, 1000), upright and fitted:

```
«component (-1000,1000) 0 0
  «symbol R
    «type: R»
    «description: Resistor(USA Style Symbol)»
    «shorted pins: false»
    «line (0,200) (0,180) 0 0 0x1000000 -1 -1»
    «line (0,-180) (0,-200) 0 0 0x1000000 -1 -1»
    «zigzag (-80,180) (80,-180) 0 0 0 0x1000000 -1 -1»
    «text (100,150) 1 7 0 0x1000000 -1 -1 "R1"»
    «text (100,-150) 1 7 0 0x1000000 -1 -1 "<val>"»
    «pin (0,200) (0,0) 1 0 0 0x0 -1 "+"»
    «pin (0,-200) (0,0) 1 0 0 0x0 -1 "-"»
  »
»
```

The same resistor rotated 90° (code 2) is written with the same symbol coordinates. Only the code changes, and the attribute text positions and alignments, which QSpice's built-in symbols adjust per rotation:

```
«component (1200,1000) 2 0
  ...
    «text (80,0) 1 110 0 0x1000000 -1 -1 "R3"»
    «text (-80,0) 1 109 0 0x1000000 -1 -1 "<val>"»
```

```
  ...
```

Rotated built-in symbols. QSpice's own symbols store the label positions it uses for each quarter turn; rotating a component in QSpice rewrites them. These positions cannot be calculated from the rotation, so a tool that rotates a component cannot reproduce them exactly. A symbol QSpice has not drawn rotated keeps its records unchanged when rotated.

Symbol

```
«symbol name
  ...
»
```

A symbol: A drawing, pins, attribute text, and properties. It is the root of a `.qsym` file and the one record inside each `component`.

Field	Type	Meaning
Name	Bare string, optional	The symbol's name, normally its file name without `.qsym`. It contains no spaces.

Once a symbol is in a schematic, its name rarely matters: The drawing is copied in full. It does matter for LOOKUP attributes, which find files in a folder named after the symbol under QSpice's `Repository` folder (see *Lookup index and pin index*).

A symbol holds the property records `type:`, `description:`, `library file:`, and `shorted pins:` (each optional), graphic records, `text` records, and `pin` records. It does not hold wires, buses, taps, net labels, junctions, or other symbols (see *Where records may appear*).

Symbol properties

These four records give a symbol's SPICE meaning. Each holds a bare string: Everything after the keyword's space to the end of the line.

type:

```
«type: R»
```

The SPICE device the symbol stands for: The letter (or letters) that begins its netlist line.

Value	Meaning
Missing, or empty	A **hierarchical block**: The symbol stands for another schematic (see *Hierarchical blocks*).
`R`, `C`, `L`, `V`, `I`, …	That SPICE device.
`QN`, `QP`, `MN`, `MP`, `JN`, `JP`, `ZN`, `ZP`	Transistors of each polarity.
`Ø(.dll)`	A component implemented by a DLL.

QSpice's Symbol Type list offers 33 values, in this order: (empty), `B`, `C`, `D`, `E`, `F`, `G`, `H`, `I`, `JN`, `JP`, `L`,

MN, MP, O, QN, QP, R, S, T, U, V, W, X, Y, ZN, ZP, Å, ¥, €, £, Ø(.dll), and ×. The last six are single Windows-1252 bytes followed, for the DLL type, by the text "(.dll)".

The list is what QSpice's editor offers, not a limit: A reader should accept any value.

Unknown: The case of the DLL type's text. QSchEdit writes Ø(.dll), as the list shows it; the parser's constant for it reads Ø(.DLL). Compare with a file QSpice saved before relying on either.

A type: record with nothing after the colon means the same as a missing record. *Unknown:* Which QSpice writes when the type is cleared in Symbol Properties. The project's notes disagree: One says QSpice writes an empty record, another that it drops the record. A reader should accept both.

description:

«description: Resistor(USA Style Symbol)»

A description of the symbol, shown in QSpice's symbol browser.

library file:

The model or subcircuit the symbol uses, in one of two forms:

- **A file name**, taken literally:
  «library file: name»
- **The subcircuit itself**, embedded. A leading vertical bar marks this form:
  «library file: |text»

In the embedded form, the text after the bar is the whole subcircuit, with each line break written as \n. QSchEdit also writes each backslash as \\. *Unknown:* Whether QSpice does the same; the parser decodes only \n.

Decode \n and \\ **only** in the embedded form. A file name can contain a backslash followed by n (C:\new\test.lib), and decoding it would break the path. *Inferred:* An unrecognized escape is kept as written (both characters).

shorted pins:

«shorted pins: false»

true or false: Whether the symbol's pins may be connected to one another without QSpice reporting it. QSpice shows this as "Allow Shorted Pins".

Electrical records

Net

`«net (x,y) size alignment kind "name" "description"»`

A net label: A name attached to whatever wire or pin meets its anchor point. The same record draws plain labels, ports, and grounds.

Field	Type	Meaning
(x,y)	Point	The anchor, where the label connects.
Size	Decimal	Font size (see *Font size*).
Alignment	Integer	Which point of the label sits on the anchor (see *Text rotation and alignment*). Labels are not rotated.
Kind	Integer	0 = a net label; 1 = a port.
Name	String	The net's name.
Description	String, optional	A line of description, drawn under the name.

Ports. A port's name is global: It connects across every schematic in a hierarchy, not just this sheet. A port is drawn as its name in a box. Otherwise it is the same as a label. *Inferred:* Any non-zero kind is a port; only 1 has been seen.

Grounds. A net named `GND`, `EARTH`, or `CHASSIS` (in any case) is drawn as a ground symbol instead of as text. Grounds do not show a description. QSchEdit does not let a ground be made a port. *Unknown:* Whether QSpice does.

Alignment. QSpice chooses the label's alignment from the wires meeting its anchor, so the label sits clear of them:

Value	Label placed
14	Above the anchor (anchor at the bottom center).
13	Below (anchor at the top center).
11	To the left (anchor at the right edge).
7	To the right (anchor at the left edge).
10	Up and to the left.
6	Up and to the right.
5	Down and to the right.
9	Down and to the left.
0	Centered: Where several wires cross with no free side.

An unconnected label is written with 14. An unconnected ground "hangs down" from its anchor, alignment 13. The rule QSpice uses to choose was measured and is summarized in *Choosing a*

label's alignment.

Examples from QSpice files:

```
«net (-900,900) 1 11 0 "N01"»
«net (-900,300) 1 11 0 "Bus[0:3]"»
```

Junction

```
«junction (x,y)»
```

A junction dot, drawn where wires connect.

Inferred from QSpice's files: QSpice writes a junction where **three or more** wire ends meet, and not where two wires simply continue. It does not draw junctions on buses.

Junction records are written by QSpice's editor as wires are drawn. A reader should draw the junctions the file holds rather than calculating its own: The file is what QSpice will show.

Wire

```
«wire (x1,y1) (x2,y2) "net name"»
```

A straight wire between two points.

Field	Type	Meaning
(x1,y1), (x2,y2)	Points	The ends.
Net name	String	The name of the net the wire belongs to.

Every wire carries its net's name. Where a net has a label, its wires carry the label's name. Where it has none, QSpice names it `N01`, `N02`, and so on, and writes that name on each of its wires. *Inferred:* Unlabeled nets are renumbered as the schematic changes, so these names are not stable.

Wires are split at connections. QSpice's editor cuts a wire wherever a connection point lands on it, so wires meet only at their ends. Two wires that cross without an end at the crossing are not connected.

Buses. A wire whose net name has the form `Name[n1:n2]` is a **bus**: A bundle of the nets `Name[k]` for every k from the smaller of n1 and n2 to the larger. The numbers are whole numbers, in either order. A bus is drawn wider than a wire.

```
«wire (-400,300) (500,300) "Bus[0:3]"»
```

Tap

```
«tap (x1,y1) (x2,y2)»
```

A bus tap: A short connector from a bus to one of its member nets. A tap is 100 units long, or 141 diagonally (100 units in each direction).

QSpice writes the **member** end first. *Inferred:* The order does not change the meaning, but a tool should keep it.

A tap is valid when one end lies on a bus (at an end or anywhere along it) and the other end reaches a member of that bus: A wire carrying a member's name, or a net label naming one. A tap never connects the bus's net to the member's net; it only shows which member is used.

Condition	Drawn as
One end on a bus, the other on a member of it	A triangle in the wire color, pointing at the member.
One end on a bus, the other on a net that is not a member	The same triangle, in the "bus tap error" color (red).
No bus, a bus at both ends, or nothing at the member end	A red spindle.

Pin

```
«pin (x1,y1) (x2,y2) size alignment pininfo color lookup "label" "net"»
```

A connection point of a symbol. Pins appear only inside a symbol. They are numbered in record order, starting at 1.

Field	Type	Meaning
(x1,y1)	Point	The connection point, in the symbol's coordinates.
(x2,y2)	Point	The label's offset from the connection point.
Size	Decimal	Font size of the label.
Alignment	Integer	The label's alignment and rotation (see *Text rotation and alignment*).
Pin info	Integer	Port and data type, for DLL components (below). 0 for an ordinary pin.
Color	Color	The label's color.
Lookup	Integer	*Inferred:* Lookup index (see *Lookup index and pin index*).
Label	String	The pin's name, as shown and as used by DLL components.
Net	String, optional	A net the pin is connected to **inside** the symbol.

A pin's lead is not part of the `pin` record. QSpice's symbols draw it as a separate `line`.

Pins sit on the 100-unit grid. Symbols place pins on multiples of 100 in their own coordinates, which keeps them on the grid at every rotation.

The label is hidden when the alignment's lower four bits are 0 or 15 (centered both ways), or when the pin carries a net name. Ordinary component pins are written with alignment 0, so their names are not shown.

A pin with a net name is connected to that net inside the symbol and draws nothing: No label and no connection mark. QSpice uses this for internal connections, such as one output of a latch symbol.

A disabled pin has the net name `¥` (the single Windows-1252 byte `0xA5`). *Inferred:* It and every graphic associated with it (see *Lookup index and pin index*) are hidden.

Pin info describes a DLL component's port. *Inferred* from QSpice's symbol editor:

Bits	Meaning
0–3	Port type: 1 = input; 2 = output; 3 = DLL ground.
4–7	Data type (table below).
8–15	First bit-vector bound.
16–23	Second bit-vector bound.

Data type	Meaning
1	`bool`
2	`char`
3	`unsigned char`
4	`short`
5	`unsigned short`
6	`int`
7	`unsigned int`
8	`float`
9	`double`
10	`int64`
11	`unsigned int64`
12	Bit vector

A port type or data type of 0 means "not set".

A resistor's pin, from a QSpice file, with its label hidden (alignment 0):

```
«pin (0,200) (0,0) 1 0 0 0x0 -1 "+"»
```

A pin whose label shows, starting 10 units to the right of the pin (alignment 7, west):

```
«pin (-200,0) (10,0) 0.681 7 0 0x1000000 -1 "EN"»
```

Graphic records

Graphics draw lines and shapes. They appear in schematics and in symbols. In a symbol, they take part in the symbol's rotation.

All graphic records end with the same group of fields:

Field	Type	Meaning
Width	Integer	Line width (see *Line width*).
Type	Integer	Line type (see *Line type*).
Color	Color	Line color (see *Colors*).

Field	Type	Meaning
Fill	Color	Fill color, for closed shapes only.
Lookup	Integer	Lookup index (see *Lookup index and pin index*).
Pin	Integer	Pin index (see *Lookup index and pin index*).

Line

```
«line (x1,y1) (x2,y2) width type color lookup pin»
```

A straight line between two points.

```
«line (0,200) (0,180) 0 0 0x1000000 -1 -1»
```

Box-defined shapes

`rect`, `ellipse`, `arc`, `coil`, and `zigzag` are each defined by a **box**: Two opposite corners and a rotation code.

The corners are opposite corners of a box in a frame **rotated by the code × 45°**, about the box's center (the midpoint of the two corners). Rotating a box by a multiple of 90° leaves it the same box, so only odd versus even matters for most shapes:

- **Even codes** (0, 2, 4, 6): The box is upright, with the corners as written.
- **Odd codes** (1, 3, 5, 7): The box is tilted 45°. Its sides run diagonally, and its shape differs from the upright box with the same corners.

The coil is the exception: Its loops depend on the full code and on the order of the corners (see *Coil*).

All sixteen codes were measured on the arc, and codes 0 to 7 on rectangles, ellipses, image rectangles, and both arcs. *Unknown:* QSpice draws odd codes, but its editor offers no way to set one on a graphic. The mirror bit's effect on a coil has been measured only at codes 0 and 8.

Rect

```
«rect (x1,y1) (x2,y2) rotation width type color fill lookup flag pin image»
```

A rectangle.

Field	Type	Meaning
Corners	Points	Opposite corners (see *Box-defined shapes*).
Rotation	Integer	Rotation code.
Width, type, color, fill		As for all graphics.
Lookup	Integer	Lookup index.
Flag	Integer	1 on the body of a hierarchical block's symbol, 0 otherwise.

Field	Type	Meaning
Pin	Integer	Pin index.
Image	Hexadecimal, optional	Image data (see *Image rectangles*). Present only for an image.

The lookup index comes **before** the flag, and the pin index after it: A different order from the other graphics.

A block's body. When the flag is 1, QSpice draws the rectangle in its own colors for hierarchical blocks (gray, with a blue border by default) and ignores the record's line and fill colors.

```
«rect (-1400,1600) (-500,800) 0 0 0 0x4000000 0x1000000 -1 0 -1»
```

Image rectangles

A `rect` whose fill mode is `0x05` to `0x0B` draws an image, stretched to fill the rectangle. The image is the record's last field: The bytes of an image **file**, written as hexadecimal digits, with no spaces or line breaks.

QSpice stores whatever format the image came in. PNG (beginning `89504e47`) and BMP (beginning `424d`) have both been seen.

This field is often hundreds of kilobytes on one line. A reader must not assume lines are short.

The image is stretched to the box. With an odd rotation code it tilts 45° with the box, but it never turns further with the code, and it does not flip with the corner order. QSpice draws the outline over the image, in the record's line color and type; its own images use a dotted outline.

```
«rect (-670,239) (-430,461) 0 0 2 0xff0000 0x5000000 -1 0 -1 424db640...»
```

Ellipse

```
«ellipse (x1,y1) (x2,y2) rotation width type color fill lookup pin»
```

The ellipse filling a box (see *Box-defined shapes*).

```
«ellipse (-130,130) (130,-130) 0 0 0 0x1000000 0x1000000 -1 -1»
```

Arc3p

```
«arc3p (x1,y1) (x2,y2) (cx,cy) width type color lookup pin»
```

The three-point arc, which QSpice's editor draws with Ctrl+A. It is not an arc of a circle through three points.

- The **third** point is the center.
- The curve runs **counter-clockwise** from the direction of the first point to the direction of the second, as seen from the center.
- Its distance from the center changes **steadily with the angle**, from the first point's distance at the start to the second point's at the end.

When the first two points are the same distance from the center, this is an ordinary circular arc.

When they are not, it is a spiral.

```
«arc3p (0,0) (60,0) (30,0) 0 0 0x1000000 -1 -1»
```

Arc

```
«arc (x1,y1) (x2,y2) (x3,y3) (x4,y4) rotation width type color lookup pin»
```

The four-point arc: Part of an ellipse.

Field	Meaning
(x1,y1), (x2,y2)	Opposite corners of the ellipse's box (see *Box-defined shapes*).
(x3,y3)	The start: A direction from the box's center. Only its direction matters, not its distance.
(x4,y4)	The end, in the same way.
Rotation	The box's rotation code.

The arc is drawn **counter-clockwise** from the start direction to the end direction. When the start and end are in the same direction, nothing is drawn (unlike the Windows `Arc` function, which would draw the whole ellipse).

QSpice reads and draws this record, but its editor does not appear to create one.

```
«arc (100,-100) (-100,-50) (-100,-75) (100,-75) 0 0 0 0x1000000 -1 -1»
```

Triangle

```
«triangle (x1,y1) (x2,y2) (x3,y3) width type color fill lookup pin»
```

A triangle through three points. It has no rotation code.

```
«triangle (0,0) (60,0) (30,0) 0 2 0xff0000 0x2000000 -1 -1»
```

Coil

```
«coil (x1,y1) (x2,y2) rotation width type color lookup pin»
```

A row of inductor loops, filling a box. It has no fill.

The coil's shape was measured from QSpice drawings and is complicated. In outline:

- The loops run along the box's **longer** side. A square box (or a tie) runs vertically.
- **The order of the corners matters.** The same box written from the other pair of corners puts the loops on the other side. The mirror bit reverses the side again.
- For an odd code, the frame is turned by the code × 45° (not just 45° as for other shapes), so the full code matters.
- The number of loops grows with the box's proportions. With r = long side ÷ short side, the measured counts are:

R	**1.5**	**1.9**	**2**	**2.5**	**3**	**3.5**	**4**	**5**	**6**	**7**
Loops	1	1	2	2	3	4	5	6	8	9

These fit loops = max(1, ⌊1.5 r − 0.875⌋). *Unknown:* Counts outside r = 1.5 to 7.

The exact curve our programs draw, which matches every QSpice coil tested, is in

`SchItemCoil.cpp` in the QSch_Shared_Libs.

```
«coil (450,-1400) (1100,-1700) 0 0 2 0xff0000 -1 -1»
```

Zigzag

```
«zigzag (x1,y1) (x2,y2) rotation width type color lookup pin»
```

A resistor's zigzag, filling a box and running along its longer side. It has no fill, and it looks the same turned end to end.

The number of peaks grows with the box's proportions. With r = long side ÷ short side, the zigzag has p pairs of peaks (2p + 1 segments):

R	**1.5**	**2**	**2.25**	**2.5**	**3**	**3.5**	**4**	**5**	**6**	**7**	**8**
Pairs	2	2	2	2	3	3	4	4	5	6	7

These fit p = max(2, ⌊0.75 r + 1.0625⌋). *Unknown:* Counts outside r = 1.5 to 8.

```
«zigzag (-80,180) (80,-180) 0 0 0 0x1000000 -1 -1»
```

Text

```
«text (x,y) size alignment flags color lookup pin "text"»
```

One record serves for SPICE directives, comments, and a symbol's attribute text.

Field	**Type**	**Meaning**
(x,y)	Point	The anchor.
Size	Decimal	Font size (see *Font size*).
Alignment	Integer	Which point of the text sits on the anchor, and its rotation (see *Text rotation and alignment*).
Flags	Integer	Bit 0: Set for a comment, clear for a directive. Bit 1: Set to hide the text.
Color	Color	Text color.
Lookup	Integer	Lookup index.
Pin	Integer	Pin index.
Text	String	The text (see *Text encoding*, *Line breaks in text*, and *Overbars*).

On a schematic

A `text` record on the schematic itself is either:

- **A SPICE directive** (flags bit 0 clear): Text passed to the simulator, such as `.tran 10m`. It may hold several lines.
- **A comment** (flags bit 0 set): Text for people, ignored by the simulator.

The hidden bit has no effect here. *Inferred:* QSpice always shows schematic text.

Each line of multi-line text is aligned by the anchor's side: Lines are left-aligned when the anchor is on the west side, right-aligned when it is on the east side, and centered otherwise. *Unknown:* This is our programs' interpretation; it has not been compared closely with QSpice.

In a symbol

Inside a symbol, `text` records are the symbol's **attributes**, identified by position:

Position	**Meaning**
First	The reference designator, such as `R1`.
Second	The value, such as `10K`. QSpice's symbols use the placeholder `<val>`.
Later	Further attributes, such as model parameters. *Inferred:* Each is written into the component's netlist line.

The hidden bit (bit 1) hides an attribute. *Inferred:* A hidden attribute is still used in the netlist.

In a symbol file, the designator is a placeholder. QSpice names a placed component with the placeholder's letters and the lowest free number for them in the schematic, so a placeholder `R5` becomes `R1` in an empty schematic.

Attribute labels stay readable. Attribute text on a mirrored component is never drawn upside down or backward. When the text would read right to left, QSpice turns it half a turn about its anchor; the box then extends to the other side of the anchor. This was confirmed on all sixteen orientations of a resistor. *Unknown:* Whether QSpice also turns upside-down text on a component that is not mirrored, and which way exactly vertical text reads.

Connectivity

QSpice connects things by **exact coincidence**: Two things are connected when they share a point, with identical integer coordinates. Nothing connects by being near.

The connection points are:

- Both ends of every wire and bus.
- Every pin of every component, at its placed position (see *Rotation code*).
- Every net label's anchor.
- Both ends of every tap, for showing the tap's state only. A tap never joins nets.

A **net** is a group of points joined by wires, together with everything that touches those points. Two nets are also joined when they carry the same label name, anywhere on the sheet. Port names join across every sheet of a hierarchy.

Names are compared **without regard to case**: `Vout` and `VOUT` are the same net, and so are the designators `r1` and `R1`.

Buses. A plain wire touching a bus joins the bus's net. Taps show a member's connection to a bus

but do not create it; members connect by name.

Unconnected marks. QSpice draws a small box on a component pin with nothing else at its point. Another pin at the same point (of the same component or another) counts as a connection.

Choosing a label's alignment

When a net label's anchor lands on wires, QSpice sets its alignment so the label sits clear of them. Measured and matched on 42 of 42 labels:

1. Add up the directions of the wires leaving the anchor, as unit vectors.
2. **If the sum is not zero**, place the label on the opposite side, snapped to the nearest of above, below, left, and right. A tie goes to above or below. This covers a single wire and an L-shaped corner.
3. **If the sum is zero and there are two wires** (a straight wire through the anchor), place the label on the upward side: Above a horizontal wire, to the right of a vertical one, or at the upper corner beside a diagonal one.
4. **If the sum is zero and there are three or more wires**, use the first corner with no wire through it, in the order up-right, up-left, down-right, down-left. If every corner has a wire, use 0 (centered).

A tap counts as a wire for this purpose.

Unknown: Whether QSpice recalculates alignments when it opens a file. A reader should draw the alignment the record holds.

Hierarchical blocks

A component is a **hierarchical block** when its symbol's `type:` record is missing or empty. A block stands for another schematic, which QSpice netlists in its place.

The schematic's file name is the block's **first string attribute** (its second `text` record, the first after the designator) with `.qsch` added. It may be an absolute path, or relative to the folder of the schematic holding the block.

Inferred: This is how our programs find a block's schematic, and they accept either `\` or `/` between folders. How QSpice itself resolves the name has not been compared.

Inferred: The block's pins connect to the ports of the same names in the child schematic.

The block's body is a `rect` with its flag set to 1 (see *Rect*).

Symbol files

A `.qsym` file holds one symbol, written exactly as a `symbol` record is written inside a component, as the root of the file:

```
«symbol name
```

```
  «type: ...»
  «description: ...»
  ...graphics...
  ...text...
  ...pins...
»
```

- **The header is optional.** QSpice writes `FF D8 FF DB` when it saves a symbol, but the symbol files installed with QSpice begin directly with `«symbol`.
- **Coordinates are the symbol's own.** The origin is the point placed at the component's position. Pins should be on multiples of 100.
- **Attribute text** follows the rules in *In a symbol*. The designator is a placeholder.
- When QSpice places a symbol from a file, it copies the whole symbol into a new `component` record, so later changes to the file do not affect schematics that already use it.

Where records may appear

The format allows any record inside any container. QSpice's editors create only these combinations:

Container	Holds	Never holds
Schematic (root)	Components, net labels, junctions, wires, taps, graphics, and text.	Pins, bare symbols, or another schematic.
Component	Exactly one symbol.	A second symbol, components, or schematics.
Symbol	Properties, graphics, text, and pins.	Wires, buses, taps, net labels, junctions, symbols, components, or schematics.

Our programs leave out any record found where QSpice's editors never create it, and say so.
Unknown: What QSpice does with such a file.

Copying and pasting

QSpice's clipboard holds **loose records**: No file header and no `«schematic»` root, just the records, one per line. Within each kind, QSpice puts them on the clipboard in the reverse of file order.

Unknown: QSpice also accepts a pasted complete document, as our programs write, but whether that is reliable has not been tested widely.

Open questions

Question	What is known
Is record order required?	QSpice always writes the fixed order. Files out of order have not been tested.
Color modes `0x02` and `0x03` as line colors	Never seen.

Question	What is known
Color mode `0x02` as a fill	Seen on triangles; assumed to be the background color.
Color mode `0x03` as a fill	Solid; whether in the drawing color or the shape's own line color.
Text rotation codes 8–15	Allowed by the field; never seen.
Pin info values	Taken from QSpice's symbol editor; not tested against the netlist.
The DLL type's spelling	`Ø(.dll)` or `Ø(.DLL)`.
A bare `0xAC` as an overbar marker	QSpice writes UTF-8; the single byte has not been tested.
Line thickness in diagonal symbols	Whether it is enlarged by √2 with the drawing.
Coil and zigzag counts	Measured for proportions from 1.5 to 7 or 8 only.
LOOKUP attributes	Not studied in detail.
Clearing a symbol's type	Whether QSpice writes an empty `type:` record or drops it.
Backslashes in an embedded subcircuit	Whether QSpice escapes them as `\\`.
A hierarchical block's file name	How QSpice resolves a relative name.
Upside-down attribute text	Whether QSpice turns it on components that are not mirrored.
Records in unexpected containers	What QSpice does with them.

Appendix A: A complete example

A small schematic, written by hand to show the main parts of the format: A voltage source and a resistor in a loop, with ground, a labeled node, and a simulation directive. The symbols are shortened to their essentials, so it is not exactly what QSpice would write for these parts.

```
FF D8 FF DB  (the four header bytes)
«schematic
  «component (0,0) 0 0
    «symbol V
      «type: V»
      «description: Independent Voltage Source»
      «shorted pins: false»
      «line (0,200) (0,130) 0 0 0x1000000 -1 -1»
      «line (0,-130) (0,-200) 0 0 0x1000000 -1 -1»
      «ellipse (-130,130) (130,-130) 0 0 0 0x1000000 0x1000000 -1 -1»
      «text (150,100) 1 7 0 0x1000000 -1 -1 "V1"»
      «text (150,-100) 1 7 0 0x1000000 -1 -1 "5"»
      «pin (0,200) (0,0) 1 0 0 0x0 -1 "+"»
      «pin (0,-200) (0,0) 1 0 0 0x0 -1 "-"»
    »
  »
  «component (800,0) 0 0
    «symbol R
```

```
      «type: R»
      «description: Resistor»
      «shorted pins: false»
      «line (0,200) (0,180) 0 0 0x1000000 -1 -1»
      «line (0,-180) (0,-200) 0 0 0x1000000 -1 -1»
      «zigzag (-80,180) (80,-180) 0 0 0 0x1000000 -1 -1»
      «text (100,150) 1 7 0 0x1000000 -1 -1 "R1"»
      «text (100,-150) 1 7 0 0x1000000 -1 -1 "1K"»
      «pin (0,200) (0,0) 1 0 0 0x0 -1 "+"»
      «pin (0,-200) (0,0) 1 0 0 0x0 -1 "-"»
    »
  »
  «net (400,400) 1 14 0 "Vin"»
  «net (0,-400) 1 13 0 "GND"»
  «wire (0,200) (0,400) "Vin"»
  «wire (0,400) (400,400) "Vin"»
  «wire (400,400) (800,400) "Vin"»
  «wire (800,400) (800,200) "Vin"»
  «wire (0,-200) (0,-400) "GND"»
  «wire (0,-400) (800,-400) "GND"»
  «wire (800,-400) (800,-200) "GND"»
  «text (-200,-700) 1 7 0 0x1000000 -1 -1 ".tran 10m"»
»
```

Points to notice:

- The groups come in the fixed order: Components, then net labels, junctions, wires, and finally text.
- R1's upper pin is at (800, 0) + (0, 200) = (800, 200), where a wire ends, so they connect.
- The label "Vin" sits on the wire at (400, 400). The top wire is split there, so the label's anchor is a wire end. Two wires continue straight through, so the label goes above (14).
- There is no `junction` record: At every corner only two wire ends meet. A third wire ending at any of those points would call for one.
- Every wire carries its net's name.

Appendix B: Record summary

Record	Fields
`schematic`	(none) — container
`component`	(x,y) rotation stuffing — container
`symbol`	Name — container
`type:`	Text
`description:`	Text
`library file:`	Text: A file name, or a vertical bar and an embedded subcircuit
`shorted pins:`	`true` or `false`
`net`	(x,y) size alignment kind "name" "description"?
`junction`	(x,y)
`wire`	(x1,y1) (x2,y2) "net"

Record	Fields
`tap`	(x1,y1) (x2,y2)
`line`	(x1,y1) (x2,y2) width type color lookup pin
`rect`	(x1,y1) (x2,y2) rotation width type color fill lookup flag pin image?
`ellipse`	(x1,y1) (x2,y2) rotation width type color fill lookup pin
`arc3p`	(x1,y1) (x2,y2) (center) width type color lookup pin
`arc`	(corner) (corner) (start) (end) rotation width type color lookup pin
`triangle`	(x1,y1) (x2,y2) (x3,y3) width type color fill lookup pin
`coil`	(x1,y1) (x2,y2) rotation width type color lookup pin
`zigzag`	(x1,y1) (x2,y2) rotation width type color lookup pin
`text`	(x,y) size alignment flags color lookup pin “text”
`pin`	(x1,y1) (x2,y2) size alignment pininfo color lookup “label” “net”?

A `?` marks an optional field.

Appendix C: Sources

This document draws on the code of The QParser2 Project, where each finding is recorded beside the code that uses it:

Topic	Where
File structure, records, and fields	QParser2_Shared_Lib: `ItemTreeIO`, `ItemTypes.h`, `ArgUtils2.h`, and each `Item*.h`
Rotation codes and matrices	SchCore: `SchRotDecode.h`
Text encoding and line breaks	SchCore: `SchStringDecode.h`
Buses	SchCore: `SchBus.h`
Connectivity and junctions	SchCore: `SchNetModel`, `SchConnPlan`
Hierarchical blocks	SchCore: `SchHierBlock`
Record placement	SchCore: `SchRecordRules`
Colors, lines, and fonts	SchRender: `SchColorDecode.h`, `SchPen.h`, `SchTextLayout.h`
Shapes	SchRender: `SchBoxFrame.h`, `SchItemArc3P`, `SchItemArc4P`, `SchItemCoil`, `SchItemZigzag`, `SchItemRect`
Symbol types, library files	QSchEdit: `SymPropsPnl.cpp`,

Topic	Where
	`SchLibFileDlg`

The full sources are at https://github.com/robdunn4/QSpice.